

MKY FORWARDING · BOOKING ASSISTANT

Technical Documentation

Architecture, services, data model, conversation engine, document pipeline, deployment and operations.

Version	16 September 2026 · commit <code>3e12bcc</code>
Repository	github.com/shanto173/MKC-GLOBAL · branch <code>main</code> · application in <code>web/</code>
Production	https://mkc-global.vercel.app (Vercel, region Frankfurt)
Database	Supabase project <code>kverwgcgvnferjewvvyr</code> (Postgres, AWS eu-central-1 Frankfurt)
Tests	237 automated tests, no network, no API keys — <code>npm test</code>

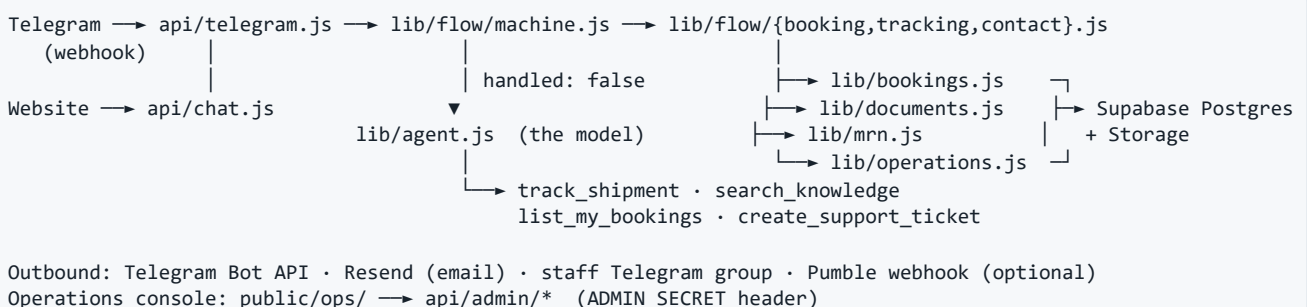
Contents

1. Overview and architecture
2. Services and technology stack
3. Repository layout
4. Deployment: GitHub → Vercel
5. Configuration: environment variables and runtime settings
6. Database: tables, functions, views, storage
7. The conversation engine (state machine)
8. The booking flow in detail
9. The document pipeline
10. Submission, notifications and the outbox
11. Tracking
12. Talk to an agent, hours and tickets
13. The AI's role and limits
14. The Operations console and admin API
15. Security
16. Testing and scripts
17. Performance notes
18. Known limitations and open decisions
19. Change log, 16 September 2026

1. Overview and architecture

The assistant is a Telegram bot (with the same engine behind a website chat widget) for a freight forwarder importing used commercial vehicles into Egypt. It takes booking requests in three steps, collects and reads the customs papers, tracks shipments from the company's own data, and routes anything else to a person. One rule governs the design:

The model may read; it may not decide. A language model reads a chassis number out of a sentence, reads a scanned invoice, and answers questions from the knowledge base. Whether a vehicle may be booked, what is missing, whether a request goes to Operations, and what the client is told are decided by code from rows in Postgres — and are tested without any model in the loop.



A message meets the deterministic state machine first. The machine handles commands, tapped buttons, files, shared contacts, and text that answers a question it asked. Anything else — a free question — it declines, and the model answers from the knowledge base without touching the flow's state.

2. Services and technology stack

LAYER	WHAT	NOTES
Runtime	Node.js $\geq$ 20, ES modules, no web framework	Plain <code>fetch</code> everywhere; the Telegram, OpenAI and Resend APIs are called directly.
Hosting	Vercel Serverless Functions (Hobby plan)	Region <code>fra1</code> (Frankfurt), 60 s max duration, 1024 MB. Hobby allows 12 functions per deployment; the project is at exactly 12, which is why <code>api/admin/ops.js</code> multiplexes four handlers behind rewrites in <code>vercel.json</code> .
Database	Supabase Postgres (AWS eu-central-1)	Accessed with the service-role key from the server only; Row Level Security is enabled on every table with no policies, so the public anon key reads nothing.
File storage	Supabase Storage, bucket <code>booking-docs</code>	Every uploaded document, keyed by chat / client / booking.
Messaging	Telegram Bot API, webhook mode	Webhook at <code>/api/telegram</code> , verified with <code>X-Telegram-Bot-API-Secret-Token</code> ; subscribed to messages and <code>callback_query</code> .
Language model	OpenAI — <code>gpt-4.1</code> (conversation), <code>gpt-4.1-mini</code> (extraction, document reading, vision)	<code>LLM_PROVIDER=anthropic</code> switches the conversation model to Claude. Embeddings for the knowledge base are always OpenAI <code>text-embedding-3-small</code> .
PDF reading	<code>pdfjs-dist</code> (text layer), <code>@napi-rs/canvas</code> (rendering scans for vision)	Loaded lazily, only when a file arrives.
PDF writing	<code>pdfkit</code> + <code>bidi-js</code>	Bilingual booking request / confirmation PDFs, Arabic shaped correctly. Loaded lazily.
Email	Resend	Client copy (when an email is on file) and the Operations inbox, PDF attached.
Background work	<code>@vercel/functions</code> <code>waitUntil</code>	Keeps a function alive after it has answered Telegram, so files are read in parallel.
Source control / CI	GitHub (<code>shanto173/MKC-GLOBAL</code>) → Vercel Git integration	Every push to <code>main</code> builds and deploys production.
Migrations	Supabase CLI, <code>supabase/migrations/*.sql</code>	Idempotent; <code>npm run db:push</code> .

3. Repository layout

```
web/
  api/
    telegram.js      Telegram webhook: verify, claim, acknowledge, run the flow, deliver
    chat.js          JSON endpoint for the website widget (same state machine)
    health.js        config + database self-check; reports the deployed commit
    status.js        public status
    admin/bookings.js the Operations queue: review, confirm, reject, ask the client
    admin/ops.js     tasks, MRN, outbox drain, read-test (one function, four handlers)
    admin/shipments.js · customers.js · tickets.js · users.js · label.js · setup.js
  lib/
    flow/           THE STATE MACHINE
      machine.js    one place where every transition is chosen
      states.js     every state, in one list, grouped by step
      store.js      conversation_sessions: state that survives a restart
      booking.js    steps 1-3, from the name to the yes
      tracking.js   lookup with an ownership rule
      contact.js    talk to an agent: hours, urgency, tickets
      keyboards.js  inline keyboards and callback payloads (≤ 64 bytes)
      messages.js   every word said to a client, Arabic and English
      paste.js      readers for pasted tables and labelled rows
      understand.js pattern reader: what a message is, and what it names
      nlu.js        the model as a second reader (extracts, never decides)
      ready.js      is the schema there? degrade instead of dying
    bookings.js    rules: duplicates, required fields, drafts, submission
    documents.js   record, store, read, classify, cross-check; batches
    phone.js       one stored shape for a phone number
    clients.js     Telegram identity, the number on file, ownership
    settings.js    bot_settings: business values changed without a deploy
    background.js  deferred work, flushed before the function returns
    outbox.js      nothing important is sent from inside a handler
    operations.js  the work queue + OperationsNotifier
    notify.js · pdf.js · storage.js · read-file.js · extract.js · mrn.js
    agent.js · tools.js · llm.js · session.js · audit.js · pinned.js
    admin/        the console's handlers (console.js, tasks.js, mrn.js, read-test.js)
  public/index.html website chat widget
  public/ops/     the Operations console (index.html, app.js, workflow.js)
  supabase/migrations/ the whole database, idempotent, CLI-pushable
  scripts/       seed, ingest, webhook setup, demo documents, smoke tests
  tests/         237 tests over an in-memory Postgres double
  docs/          SETUP, ARCHITECTURE, DEMO-DATA, HOW-WE-BUILT-THIS, ROTATE-CREDENTIALS
```

4. Deployment: GitHub → Vercel

1. Changes are committed on `main` and pushed to `origin` (GitHub, `shanto173/MKC-GLOBAL`).
2. Vercel's Git integration builds the `web/` project and deploys it to production. A build takes about a minute.
3. `GET /api/health` reports the deployed commit, the configured services, and whether every table and function the flow needs exists — the way to confirm a deploy landed.
4. Database changes ship as files in `supabase/migrations/` and are applied separately with `npm run db:push` (Supabase CLI, linked to the project) or by pasting the SQL into the Supabase SQL editor. Migrations are written to be safe to re-run.
5. If code that needs a new table reaches production before its migration, `lib/flow/ready.js` detects the missing table and the previous engine answers instead of every message failing. The moment the migration is applied, the flow switches on with no redeploy.

6. The Telegram webhook is registered once per deployment URL: `npm run setup:webhook -- https://mkc-global.vercel.app` (or `/api/admin/setup`). It subscribes to `callback_query`, without which inline buttons are dead.

Environment variables live in Vercel → Project → Settings → Environment Variables and take effect on the next deployment. Business values that must change without a deployment live in the `bot_settings` table (section 5) and are picked up within a minute.

5. Configuration

5.1 Environment variables (`.env` locally, Vercel in production)

VARIABLE	PURPOSE
<code>SUPABASE_URL</code> , <code>SUPABASE_SERVICE_ROLE_KEY</code>	Database and storage. The service-role key bypasses RLS and never leaves the server.
<code>TELEGRAM_BOT_TOKEN</code> , <code>TELEGRAM_WEBHOOK_SECRET</code>	Bot identity; the secret Telegram echoes on every webhook call.
<code>LLM_PROVIDER</code> , <code>OPENAI_API_KEY</code> , <code>OPENAI_MODEL</code> , <code>ANTHROPIC_API_KEY</code> , <code>ANTHROPIC_MODEL</code> , <code>EMBEDDING_MODEL</code>	Which model answers questions and reads documents; embeddings for knowledge search.
<code>COMPANY_NAME</code> , <code>COMPANY_EMAIL</code> , <code>COMPANY_PHONE</code> , <code>REFERENCE_PREFIX</code>	Branding on PDFs and references (<code>MKY-BKG-...</code> , <code>MKY-TKT-...</code>).
<code>OPERATIONS_PHONE</code> , <code>DIRECT_PHONE</code>	The desk's number; the after-hours direct line. Both overridden by <code>bot_settings</code> . The <code>.env.example</code> placeholder is never read out.
<code>ADMIN_SECRET</code>	Protects <code>/api/admin/*</code> ; travels in a header, never in a URL.
<code>RESEND_API_KEY</code> , <code>MAIL_FROM</code> , <code>OPS_EMAIL</code>	Booking emails.
<code>STAFF_CHAT_ID</code>	Telegram group that receives each new booking PDF.
<code>BOOKING_ENGINE</code>	<code>state_machine</code> (default). <code>llm</code> restores the earlier model-driven booking as a rollback.
<code>CRON_SECRET</code> , <code>PUMBLE_ENABLED</code> , <code>PUMBLE_WEBHOOK_URL</code> , <code>APP_BASE_URL</code> , <code>BOOKING_FORM_URL</code> , <code>BOT_PROMPT_VERSION</code>	Outbox cron auth, optional chat notifications, CLI base URL, prompt traceability.

5.2 Runtime settings (`bot_settings` table, key → JSON value)

KEY	DEFAULT	MEANING
<code>required_booking_documents</code>	<code>["invoice","brief","mrn"]</code>	What a client must send when they hold the MRN.

<code>required_booking_documents_mky_mrn</code>	<code>["invoice", "brief"]</code>	Same, when MKY obtains the MRN.
<code>required_mrn_documents</code>	<code>[]</code>	Awaiting MKY. What MKY needs to apply for an MRN. While empty the bot collects a description and raises a task.
<code>acid_required</code>	<code>false</code>	Awaiting MKY. Whether ACID is required at booking time.
<code>allow_submit_while_mrn_pending</code>	<code>true</code>	Whether a request may reach Operations before the MRN is issued.
<code>allowed_file_types</code> , <code>max_upload_bytes</code>	PDF, JPEG, PNG, WebP, HEIC · 20 MB	Upload validation, before the bytes are fetched.
<code>draft_expiry_hours</code>	<code>72</code>	An untouched draft is expired, not offered back.
<code>operations_phone</code> , <code>operations_email</code> , <code>human_support_hours</code>	null · null · "Every day, 09:00-19:00 Cairo time"	Shown to a client in hours.
<code>direct_phone</code>	currently a placeholder , +48 512 345 678	Given to an after-hours client who says it is urgent. Falls back to <code>DIRECT_PHONE</code> , then the desk's number; with none, nothing is read out.
<code>support_hours_start</code> , <code>support_hours_end</code> , <code>support_timezone</code>	9 · 19 · Africa/Cairo	When a person answers. Read on every request; changes apply within a minute.
<code>allow_unowned_shipment_tracking</code>	<code>false</code>	Demo only: lets ownerless seed shipments be tracked by anyone.
<code>sla_new_request_hours</code> , <code>sla_review_hours</code> , <code>sla_mrn_hours</code>	2 · 4 · 8 (placeholders)	When the console marks work overdue. To confirm with MKY.

6. Database

6.1 Tables

TABLE	HOLDS
-------	-------

<code>clients</code>	One row per person: Telegram user id, chat id, display name, <code>phone</code> , <code>email</code> , blocked flag.
<code>bookings</code>	Booking requests: reference, status (draft → pending_review → under_review → needs_client_action → confirmed rejected cancelled expired), customer name and contact, chassis (<code>vin</code> , generated <code>vin_norm</code>), make/model, ports, MRN choice, assignment, review timestamps, ops notes, edit history, priority. A partial unique index forbids two live bookings for one chassis; a check constraint forbids a submitted booking without its basics.
<code>booking_documents</code>	Every uploaded file: type (invoice, brief, mrn, acid, eur1, other), chassis read from it, storage path, extracted fields, reading state (<code>extracted.pending</code> while being read), status (received → pending_verification → verified rejected replacement_requested), Telegram file ids, uploader.
<code>vehicles</code>	Units MKY already knows (make, model) by chassis.
<code>shipments</code> , <code>shipment_events</code>	Live shipments (status, vessel, ETA, ACID, B/L, container, route) and their event history; opened from a confirmed booking.
<code>mrn_requests</code>	MRN applications MKY makes on a client's behalf, with what the client supplied and what is missing.
<code>operations_tasks</code>	The desk's work queue: new booking, MRN request, client callback, client responded; idempotency key, priority, assignment.
<code>support_tickets</code>	Tickets from "Talk to an agent": department, contact, summary (prefixed URGENT when marked so), resolution.
<code>notification_outbox</code>	Every message that must reach a client, with an idempotency key, retry schedule and delivery state. Also records "this batch of files was answered" (event <code>document_reply</code>).
<code>conversation_sessions</code>	Where each chat is: current state, active flow, active booking, flow bookkeeping (<code>context</code>).
<code>conversations</code>	Transcript memory for the model's question answering.
<code>processed_updates</code>	Telegram update ids, claimed atomically; retries do no work twice.
<code>audit_logs</code>	Who did what to which record, redacted of secrets and document bodies.
<code>bot_settings</code>	Runtime settings (section 5.2).
<code>documents</code>	Knowledge base chunks with embeddings (pgvector) for question answering.
<code>ops_users</code> , <code>internal_notes</code>	Console users; internal notes on bookings.

6.2 Functions and views

OBJECT	ROLE
<code>claim_telegram_update(update_id, chat_id)</code>	First caller wins; a crashed worker's claim is reclaimable after two minutes.

<code>submit_booking_request(booking_ref, chat_id, task_ref)</code>	Ownership check, duplicate check, status change and Operations task in one transaction. A second caller is told <code>already: true</code> .
<code>next_shipment_id(prefix)</code>	Sequential shipment references when a booking is confirmed.
<code>find_shipments(q)</code>	Shipment search by reference, ACID, B/L, container or chassis.
<code>match_documents</code> , <code>search_documents_fts</code>	Vector search over the knowledge base, with a full-text fallback when no embedding key is configured.
<code>touch_booking_status_changed</code> , <code>touch_ticket_status_changed</code>	Triggers stamping status changes for SLA tracking.
views <code>booking_queue</code> , <code>client_request_queue</code>	The console's main screens, assembled in Postgres and paged.

7. The conversation engine

`lib/flow/machine.js` is the one place a transition is chosen, always from (current state, event). Handlers return `{ messages, patch }` and send nothing themselves; the machine applies the patch to the session and hands the messages to the transport. That is what makes the whole conversation testable without Telegram.

7.1 Order of dispatch

1. **Commands** from any state: `/start`, `/menu`, `/book`, `/track`, `/cancel`, `/help`, `/reset`.
2. **Tapped buttons** (callback payloads such as `bk:confirm`, `ct:urgent:yes`). A payload says which button; the machine decides whether that is a legal move from the actual state — stale cards are refused.
3. **A shared phone number**: the answer to step 1 when that was asked; kept and the open question re-asked during a booking; otherwise the number for a ticket.
4. **A caption** (details written on a file), read before the file itself.
5. **A file**: attached to the draft in progress, else to the request already with the desk.
6. **Text**: an answer if a question is open (`AWAITING_TEXT` states); a numbered choice standing in for a button; a phrase that starts a flow; otherwise a question for the model.

7.2 States

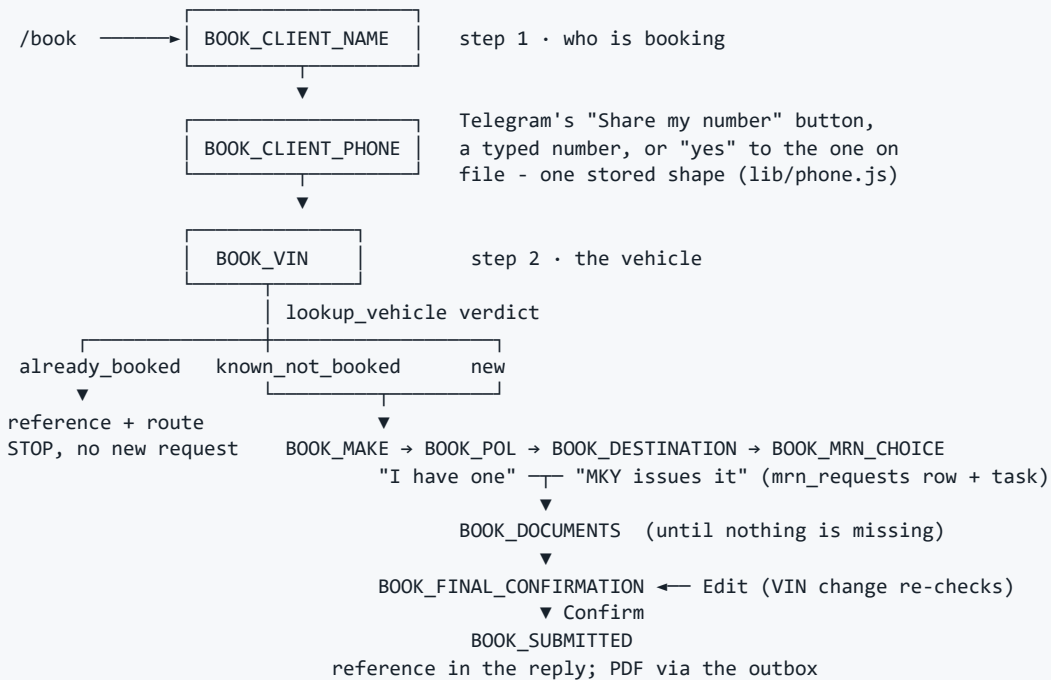
```
MAIN_MENU
Booking, step 1:  BOOK_DRAFT_RESUME · BOOK_CLIENT_NAME · BOOK_CLIENT_PHONE
Booking, step 2:  BOOK_VIN · BOOK_MAKE · BOOK_POL · BOOK_DESTINATION · BOOK_MRN_CHOICE
                  BOOK_DOCUMENTS · BOOK_DOCUMENT_CLASSIFY · BOOK_MRN_SUPPORTING_INFO
Booking, step 3:  BOOK_FINAL_CONFIRMATION · BOOK_EDIT_MENU · BOOK_CANCEL_CONFIRM · BOOK_SUBMITTED
                  BOOK_EDIT_VIN · BOOK_EDIT_MAKE · BOOK_EDIT_CLIENT_NAME · BOOK_EDIT_CLIENT_PHONE
                  BOOK_EDIT_POL · BOOK_EDIT_DESTINATION
Tracking:         TRACK_IDENTIFIER · TRACK_RESULTS
Contact:         CONTACT_URGENCY · CONTACT_TICKET_DETAILS
                  (older routes, reachable from old cards: CONTACT_MENU, CONTACT_BOOKING_IDENTIFIER,
                   CONTACT_TRACKING_IDENTIFIER, CONTACT_DOCUMENT_MENU, CONTACT_DOCUMENT_REQUEST)
```

Session state lives in `conversation_sessions`, never in memory: two serverless invocations a second apart are different processes. A state written by an older deployment that the code no longer knows falls back to the menu. A turn that says nothing and changes nothing writes nothing back.

7.3 Both channels

Telegram draws inline keyboards. The website widget receives the same choices as `options` and draws its own buttons; either way a client may type the number of a choice, except while a question is open, when a number is the answer to it.

8. The booking flow in detail



8.1 Step 1 — who is booking

- Order of the basics (`BASIC_REQUIRED` in `lib/bookings.js`): `customer_name`, `customer_contact`, `vin`, `make`, `origin_port`, `destination_port`. `missingBasics()` drives which question is next; the contact counts only when it is a dialable number, never the chat's own routing address.
- Phone numbers are normalised to one shape by `lib/phone.js`: Arabic-Indic digits converted, spaces and punctuation removed, `00` → `+`, 8–15 digits. A shared Telegram contact gets its `+` restored.
- The number is stored on the booking (`customer_contact`) and on the client (`clients.phone`), so the next booking offers it back and the next ticket does not ask.
- A message carrying both name and number ("Ariful and my number is +49 ...") is read for both: the contact is removed along with the words around it, and what remains, if it fits the question, is the answer.

8.2 Step 2 — the vehicle

- `lookupVehicle()` queries `vehicles`, `live bookings` and `shipments` by `vin_norm` and returns `new`, `known_not_booked` or `already_booked`. The check runs wherever a chassis appears — the chassis question, a pasted block at the name step, a sentence at the make step, a caption on a file — through one function, `settleVin()`. A chassis is never written down unchecked.
- Duplicate prevention is enforced three times: the flow refuses; `submit_booking_request()` re-checks inside the transaction; the partial unique index `bookings_live_vin_unique` makes it impossible at the database level.
- The destination must match one of the five Egyptian ports (aliases in Arabic and English; "Damietta port" is not Port Said). Makes are canonicalised and split from models; Arabic makes and cities are transliterated for the paperwork; a client's company name is left as written.

8.3 Reading what people actually type

Two readers, in order. `lib/flow/paste.js` and `lib/flow/understand.js` hold fixed patterns: labelled rows and pasted tables in any copy shape (colons, pipes, backticks, row numbers glued on), a chassis anywhere in a sentence, "from X to Y", a make and model mid-sentence, "client ...", phone numbers and emails, questions, refusals, requests to be elsewhere. Only when the patterns leave something missing and the message is long enough to be saying more does `lib/flow/nlu.js` ask the model to extract candidate fields — and every candidate goes through the same validation as a typed value. A wrong model answer costs at most one clarifying question, never a wrong port on a declaration.

8.4 Drafts

- One draft per chat. A Book tap with a meaningful draft open offers Continue / Start over; a blank draft is reused silently; a draft untouched for `draft_expiry_hours` is marked expired.
- Start over and Cancel drop every draft in the chat, so an older abandoned attempt cannot resurface as "an unfinished request".
- Every answer is written to the draft as it arrives; a restart, a redeploy or a second worker continues the same booking.

8.5 Step 3 — the card and the yes

The card is built from the row, never composed by a model. Confirm re-checks the basics and the documents, then calls `submit_booking_request()`. The reply carries the booking reference; the PDF copy is queued in the outbox with its own idempotency key and delivered right after (section 10). The reference stays on the session so a second tap on the same card says "already with Operations" instead of apologising for a fault.

9. The document pipeline

```
Telegram file update
| validate type/size (settings) → beginDocument(): row inserted, extracted.pending = true
| answer Telegram 200 immediately; the rest runs under waitUntil
| caption? → runFlow({kind:'caption'}) reads it onto the draft (patterns, then model)
| downloadFile → completeDocument():
|   storeDocument (Storage) → readDocument (pdf.js text, or vision on a scan/photo)
|   → extractDocument (type, chassis, numbers) → row updated, pending = false
|   → attachToOpenBooking() by chassis when the row was not already attached
| media album? wait 1.5 s for siblings to record themselves
| any sibling still reading? → say nothing (the last to finish speaks for all)
| claimReply(chat, batch ids): unique row in notification_outbox – one speaker per batch
| runFlow({kind:'document', batch})
|   → "Received: ..." + "Noted from your message: ..." + the next question
```

- **Why answer Telegram first.** Telegram delivers a chat's updates one at a time and waits for each answer before sending the next. Reading a file before answering made three files sent together read one after another, each getting its own reply. Recording the file and answering at once lets the three arrive within a second and be read in parallel.
- **One reply per batch.** Readers that finish together all see "no sibling still reading", so the reply is claimed through the unique index on `notification_outbox.idempotency_key`; only the winner speaks, and the reply lists every file in the batch and every detail the caption supplied.
- **After a file** the flow asks whatever is missing first — a basic, the MRN choice, more papers — and shows the card when nothing is. A client who sends an MRN is not asked whether they have one.
- **Papers for a request already with the desk** (submitted, under review, waiting on the client, confirmed) are filed against it and acknowledged with its reference. Any inbound message from a client the desk is waiting on flips the request back to *under review* and raises a "client responded" task.
- **Wrong vehicle.** A document whose chassis differs from the booking's is neither received nor missing: it is reported once per wrong chassis, before anything else.
- **Received is not verified.** The bot may say received. Only Operations moves a document to verified, rejected or replacement requested.

10. Submission, notifications and the outbox

```
something happens → notification_outbox row → drain → Telegram / email
(one transaction)   (idempotency_key)         | inline, right after the trigger
                                                | /api/cron/outbox by hand or by cron
```

- Retries at 1, 5, 15, 60, 240 and 720 minutes, then `dead`. A client who blocked the bot is marked dead at once. `npm run outbox -- --list` shows the queue; `npm run outbox` sends it.
- On Confirm: the request PDF to the client (outbox), the Operations email with the PDF (Resend), the staff Telegram group ping, an Operations task, and optionally a Pumble message. None of these can affect whether the booking succeeded.
- On the desk's decisions: `booking_confirmed` (+ confirmation PDF and shipment reference), `booking_rejected` with the reason, `missing_information_requested`, `mrn_issued`, free-form

`operations_message`.

- A pinned status card (one per chat, edited in place) follows open bookings and shipments and is removed when nothing is in flight.

11. Tracking

`lib/flow/tracking.js` extracts every candidate identifier from the message (bare, labelled, inside a sentence, an `MKY-...` reference) and matches each exactly against `shipments` (id, booking reference, ACID, B/L, container, `vin_norm`). A shipment is visible when the chat created it or the client behind the chat owns the booking; not-found and not-yours return the same answer. Ownerless seed rows are visible only with `allow_unowned_shipment_tracking`. Refresh re-queries by the reference carried in the button.

12. Talk to an agent, hours and tickets

- `supportHours()` (`lib/settings.js`) reads `support_hours_start / _end / _timezone` and decides open or closed on an injectable clock, so the after-hours path is tested at fixed times.
- **In hours:** a `client_callback` task is logged at once; the desk's details are shown; the client is asked what they need. A number on file (`clients.phone`, else the last booking) is used, not asked for.
- **After hours:** the closing message and "Is it urgent?" (state `CONTACT_URGENCY`). Yes → the direct line if configured, else "flagged as urgent"; the ticket summary is prefixed `URGENT:` and the task is high priority; the closing message repeats the number. No → the question, and when the desk is back. A typed answer is understood, and marked urgent when it says "urgent" or "emergency".
- A ticket needs a problem and a way to reach the client; they arrive in either order, and a client who declines to give a number is reachable through the chat. Tickets land in `support_tickets` and the console's Client Requests.

13. The AI's role and limits

WHERE THE MODEL IS USED	GUARD
Answering free questions from the knowledge base (<code>lib/agent.js</code> , tools <code>track_shipment</code> , <code>search_knowledge</code> , <code>list_my_bookings</code> , <code>create_support_ticket</code>)	It has no booking tools: <code>create_booking</code> and friends are filtered out of its tool list while the state machine owns booking. A tool never offered cannot be called.
Extracting fields from a sentence the patterns could not read (<code>lib/flow/nlu.js</code>)	Runs only when something is still missing; six-second timeout; every field re-validated (chassis shape, one of five ports, length limits).
Reading documents: text layer via <code>pdf.js</code> , scans and photos via vision (<code>lib/read-file.js</code> , <code>lib/extract.js</code>)	The reader's confident type wins over a hint; an uncertain file is classified by the client, never guessed.
Knowledge base ingestion (<code>scripts/ingest.mjs</code>)	Embeddings over MKY's own PDF/Excel; full-text fallback without an embedding key.

14. The Operations console and admin API

The console at `/ops/` (and the earlier `/ops.html`) signs in with a name from `ops_users` and `ADMIN_SECRET`, sent as a header. Screens: Dashboard, Booking Queue (whose turn, next action, readiness ticklist, documents, messages, internal notes), Document Review, MRN Requests, Client Requests, Confirmed Bookings, Shipments, My Tasks. Actions: take / put back / done on tasks; verify, reject with reason, replacement request on documents; ask the client (parks the request as waiting for client); confirm (opens the shipment, tells the client with a PDF); reject with reason; record an MRN; set vessel / ETA and tell the client; resolve or reopen tickets; print a booking label PDF.

ROUTE	PURPOSE
<code>POST /api/telegram</code>	Telegram webhook.
<code>POST /api/chat</code>	Website widget: <code>{ sessionId, message action } → { reply, options, state }</code> . Public; rate-limit before advertising.
<code>GET /api/health</code> , <code>GET /api/status</code>	Self-check with deployed commit; public status.
<code>/api/admin/bookings</code> , <code>shipments</code> , <code>customers</code> , <code>tickets</code> , <code>users</code> , <code>label</code> , <code>setup</code>	The console's data and actions; webhook registration.
<code>/api/admin/ops?resource=tasks mrn read-test outbox</code>	Four handlers in one function; <code>/api/admin/tasks</code> , <code>/mrn</code> , <code>/read-test</code> , <code>/api/cron/outbox</code> are rewrites to it.

15. Security

- The service-role key is server-only; RLS is on with no policies, so a leaked anon key reads nothing.
- The webhook verifies Telegram's secret token; every update is claimed atomically; every callback payload is untrusted and checked against the real state.
- Ownership rules on bookings and shipments; not-found and not-yours are indistinguishable.
- The audit log redacts tokens, secrets, keys, file contents and anything over 500 characters.
- The bot never reads out an unconfigured or placeholder phone number, and never invents a customs requirement.
- `docs/ROTATE-CREDENTIALS.md` and `npm run rotate` cover rotating the bot token and keys.

16. Testing and scripts

`npm test` runs 237 tests with `node --test` against `tests/helpers/fake-db.mjs`, an in-memory PostgREST double that is deliberately strict about the two things the concurrency guards are built from: unique violations raise `23505`, and a conditional update that matches nothing returns nothing. The suites: `flow.test.mjs` (the whole conversation, both languages, pastes, captions, batches, drafts, hours), `operations.test.mjs`, `extract.test.mjs`, `console.test.mjs`, `pinned.test.mjs`.

COMMAND	DOES
---------	------

<code>npm run db:push · db:link</code>	Apply migrations / link the Supabase project.
<code>npm run setup:webhook -- <url></code>	Register the Telegram webhook (with <code>callback_query</code>).
<code>npm run seed · ingest · gen:data</code>	Load shipments from Excel; build the knowledge base; regenerate demo data.
<code>npm run testdocs -- <CHASSIS> "<CLIENT>" <MAKE> "<MODEL>" <CITY> <PORT> <PREFIX></code>	Generate a consistent kit of demo PDFs (invoice, CMR, MRN, ACID, wrong-chassis invoice).
<code>npm run outbox [-- --list]</code>	Send or list queued notifications.
<code>npm run smoke · chatid · test:pdf · guide</code>	Talk to the assistant from the terminal; find the staff group id; render a sample PDF; build the client guide PDF.

17. Performance notes

- The function runs in Frankfurt next to the database; from the default Washington region every database call crossed the Atlantic and a tap made eight of them.
- Nothing that does not change the reply — the button acknowledgement, the typing indicator, the audit row, the pinned card — is waited for before the reply goes out. `lib/background.js` holds that work and the transport flushes it before returning, because a promise still in flight when a serverless function answers may never complete.
- The client upsert and the "client responded" check run alongside the update claim; the draft sweep on the Book tap is gone; the assistant and the PDF libraries load only when needed.
- Settings and the schema probe are cached per warm instance for a minute. The first request after a few idle minutes still pays Vercel's cold start.

18. Known limitations and open decisions

- **Decisions awaited from MKY:** `required_mrn_documents`, `acid_required`, the real direct number, desk days off, SLA hours, whether free questions should go to an agent rather than the AI.
- Hobby plan: cold starts; no cron more frequent than daily (queued messages go out on the next client message or desk action); twelve functions.
- `/api/chat` is public and spends model credits; add a captcha, login or rate limiting before advertising it.
- Two files that finish reading within the same few milliseconds race for the reply claim; the loser is silent, so at worst one reply — never two.
- Migration `20260916120000_three_step_booking.sql` seeds the hours and direct-number settings; the same rows were written to production directly and the migration is safe to apply later.

19. Change log — 16 September 2026

COMMIT CHANGE

d06ca97	The booking in three steps (name and number first), the chassis check wherever the chassis appears, "Talk to an agent" with after-hours message, the confirmation no longer sent twice, first webhook concurrency work.
dc0a7ba	Function moved to Frankfurt; background work registry; caption reading; one reply for several files; step-2 invitation to send everything at once.
3dfb674	Name and number in one message; "Ariful is my name" stored as the name; Arabic-aware word boundaries.
0caeea8	Files answered before they are read (<code>waitUntil</code>), read in parallel; pattern readers for "X to Y", makes and clients mid-sentence; the model reads the rest when something is missing.
74d708b	One reply per batch through a claimed slot; dangling words trimmed around a contact.
62a5628	"Reading it now" removed; what the message said read back with the files.
56469d8	Start over and Cancel clear every leftover draft; blank drafts reused; draft expiry applied.
ba2e458	Papers for a request already with the desk acknowledged with its reference.
3e12bcc	After hours: "Is it urgent?" before the direct line; urgent tickets marked; prompts reworded.